



AI-Powered Intelligent Job Search and Career System

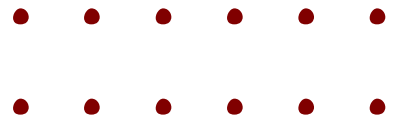
Data Science and AI Project

Milestone 4 Report

Group 4

Prepared By:

Gaurav Kumar (22f1001105)
Dev Gupta (22f2000888)
Abhinav Ohri (24f1002064)
Pranav N (22f2000117)
Praveena N (22f3001454)



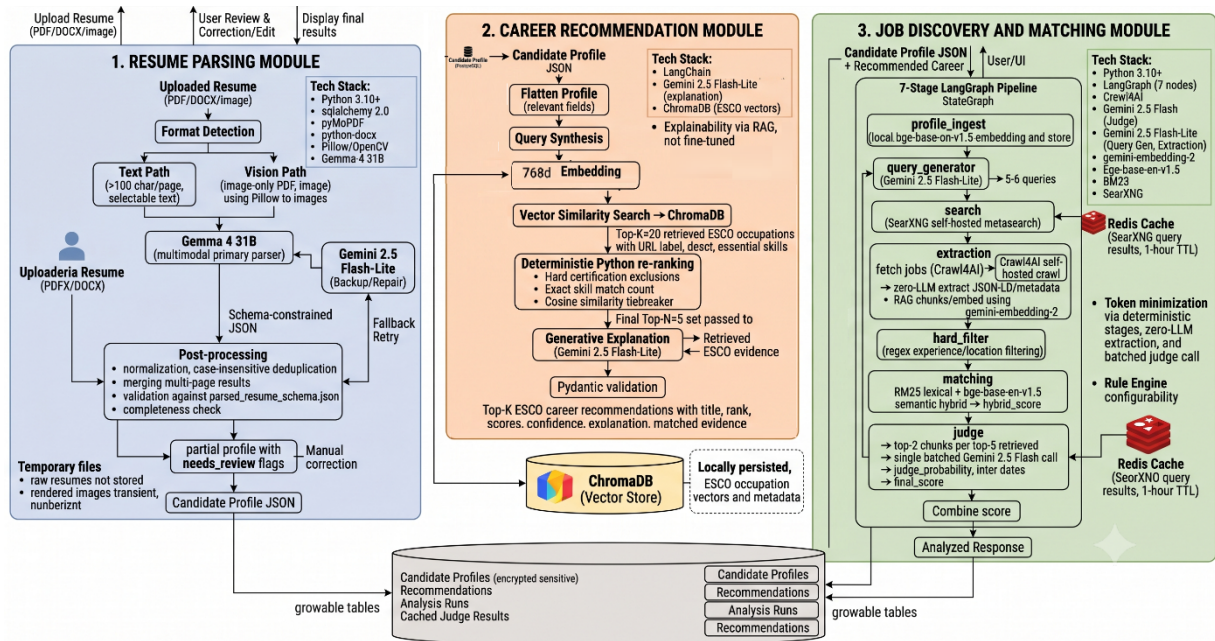


Index

SN	Particulars	Page No
1	Introduction	2
2	Datasets Used	3
3	Model Configuration	4
4	Execution Environment	5
5	Implementation Methodology	6
6	Configuration Parameters and Their Selection	7
7	Optimization Methods	8
8	Regularization and Reliability Controls	12
9	Development Progress	13
10	Final Configuration Selection	15
11	Challenges Encountered	16
12	Summary and Next Steps	18
13	Team Consent	18

1. Introduction

1.1 Model Architecture



The models used at each inference stage are:

Module	Component	Model
Resume Parsing	Primary parser	Gemma 4 31B (multimodal, open-weight)
Resume Parsing	Backup and repair	Gemini 2.5 Flash-Lite
Career Recommendation	Embedding model	BAAI/bge-base-en-v1.5, 768 dimensions, local CPU
Career Recommendation	Vector store	ChromaDB, cosine similarity
Career Recommendation	Explanation model	Gemini 2.5 Flash-Lite
Job Discovery and Matching	Query generation	Gemini 2.5 Flash-Lite
Job Discovery and Matching	Judge	Gemma 4 31B
Job Discovery and Matching	Embedding model	BAAI/bge-base-en-v1.5, 768 dimensions, local CPU
Job Discovery and Matching	Lexical ranking	BM25 (rank_bm25)

1.1 Objectives of this milestone

- Build a working prototype of each of the three modules and confirm that the end-to-end pipeline runs.
- Fix the configuration surface of each module: prompts, output schemas, retrieval parameters, and scoring weights.
- Record the datasets, splits, environment, and constraints under which the prototype operates.
- Identify what remains to be measured.

1.2 Nature of the work in this milestone

This system performs no gradient-based training. No model weights are initialised, updated, or saved at any point in the pipeline. Every model is pretrained and is used either zero-shot or few-shot through a prompt and an output schema, or as a frozen encoder that produces embeddings. This was a design decision taken because the project has no labelled training data of the kind these tasks would require.

The work of this milestone was therefore implementation rather than training. Each module was built, run on development data, and inspected.

2. Datasets Used

2.1 Datasets

Module	Dataset	Role
Resume Parsing	Resume Data (PDF), Kaggle hadikp/resume-data-pdf: 8,905 single-page image-only PDFs, 86-record gold set	Gold set supports prompt iteration and per-field measurement; the remaining corpus supports robustness sampling
Career Recommendation	ESCO v1.2.1 English classification: 3,039 occupations, 13,939 skills, 126,051 occupation-skill relations	Retrieval knowledge base, indexed in full
Job Discovery and Matching	Candidate Job Role Dataset (Kaggle): 1,000 profiles, 200-profile stratified development sample	Query side of the ranking experiments
Job Discovery and Matching	LinkedIn Job Postings Dataset (Kaggle): 123,849 postings, 24,428 companies, 72,521 titles	Offline document pool for ranking experiments
Job Discovery and Matching	Live discovery store (PostgreSQL with pgvector)	Runtime corpus populated by SearXNG discovery and Crawl4AI extraction; not used for offline experiments

2.2 Splits

Module	Split	Detail
Resume Parsing	Development 34, held-out test 52, training 0	Gold set of 86 records stratified across all 43 canonical categories. Near-duplicate groups are confined to a single split so that visually similar resumes cannot straddle the boundary. Random seed 42. The remaining 8,819 unannotated resumes support robustness sampling.
Career Recommendation	Not split	The ESCO occupation set functions as the complete retrieval knowledge base. Withholding occupations would degrade the system rather than create a meaningful held-out condition, since a candidate

Module	Split	Detail
		cannot be matched to an occupation that is absent from the index.
Job Discovery and Matching	60 / 20 / 20 on the 200-profile development sample: 120 / 40 / 40 candidates	Candidate IDs are divided before candidate-job pair construction, so every pair belonging to one candidate stays in the same split. A 60/20/20 ratio was chosen over 80/10/10 because the latter would leave only 20 candidates in each evaluation split, too few for stable ranking estimates.

2.3 Augmentation

Augmentation is used for robustness testing only, never to expand a training set, since there is none.

- **Resume Parsing:** a separate robustness set was generated using small image rotations, moderate Gaussian blur, JPEG compression, contrast reduction, resolution reduction, and light image noise. Each augmented document keeps the annotation and the split of its source resume, so a transformed copy cannot leak across the development and test boundary. Original development and test resumes are left unaltered.
- **Job Discovery and Matching:** positive candidate-job pairs are augmented through controlled skill-synonym replacement using a manually reviewed alias list, two variants per positive pair. Augmentation is applied after splitting and only within the development split; no new skills, qualifications, or job requirements are created.

All models are pretrained and used as supplied. Nothing is trained from scratch and nothing is fine-tuned. The configurable surface of the system is therefore the prompts, the output schemas, the retrieval parameters, and the scoring weights.

3. Model Configuration

3.1 Settings held constant

Setting	Value and reason
Temperature	0 for every generative call. Extraction, explanation, and judging must be reproducible; sampling variety has no value in any of these tasks.
Output format	Schema-constrained JSON on every generative call, validated before the output is accepted downstream.
Model identifiers	Held in environment configuration rather than in source, so a deprecated model can be swapped without a code change.
Random seed	42, fixed across split assignment and sampling.
Embedding dimensions	768 in both Career Recommendation and Job Discovery and Matching.

3.2 Module configuration

Module	Configuration
Resume Parsing	Primary parser: Gemma 4 31B (multimodal, open-weight). Backup and repair: Gemini 2.5 Flash-Lite, used only on fallback.

Module	Configuration
	Vision-first routing, with a 100-character-per-page text-layer probe deciding between the text path and the vision path. One page image is sent per call and the per-page JSON is merged afterwards. A single Gemini 2.5 Flash-Lite fallback retry is permitted on schema-validation failure or low field completeness, after which a partial profile with review flags is emitted rather than nothing.
Career Recommendation	Embedding model: BAAI/bge-base-en-v1.5 (768 dimensions, local CPU) Vector store: ChromaDB, cosine similarity. Explanation model: Gemini 2.5 Flash-Lite. One chunk per occupation, concatenating preferred label, alternative labels, description, and essential skills. Initial vector retrieval of Top-K 20, followed by deterministic Python re-ranking (hard-requirement exclusion, then exact skill-match count, with cosine similarity as tiebreak), reduced to Top-N 5 passed to the explanation model.
Job Discovery and Matching	Query generation: Gemini 2.5 Flash-Lite. Judge: Gemma 4 31B, Embedding model: BAAI/bge-base-en-v1.5, 768 dimensions, local CPU. Lexical ranking: BM25. Seven-stage LangGraph pipeline. Hybrid score = $(0.4 \times \text{BM25} + 0.3 \times \text{embedding}) / 0.7$, keeping the top 15. Final score = $0.8 \times \text{hybrid score} + 0.2 \times \text{judge probability on the top 5}$. Exactly two LLM calls per run regardless of how many jobs are discovered: one query-generation call to Gemini 2.5 Flash-Lite, and one batched judge call to Gemma 4 31B.

4. Execution Environment

Requirement	Resume Parsing	Career Recommendation	Job Discovery and Matching
CPU / GPU	Hosted API only (Gemma 4 31B, Gemini 2.5 Flash-Lite); no local model, no GPU.	Hosted API only (BAAI/bge-base-en-v1.5 (768 dimensions, local CPU), Gemini 2.5 Flash-Lite) ChromaDB runs on standard CPU.	Hosted API for Gemini 2.5 Flash-Lite and Gemma 4 31B bge-base-en-v1.5 (110M parameters) runs locally on CPU
External services	None beyond the hosted parser API.	ChromaDB persistent local index.	SearXNG (self-hosted) Crawl4AI (self-hosted) Redis (one-hour cache).
Storage	Structured profiles in PostgreSQL Rendered page images are transient and deleted after parsing.	ChromaDB vector index PostgreSQL for recommendation outputs.	PostgreSQL with pgvector for jobs, chunk embeddings, and run records.

4.1 Shared constraints and risks

These apply across all three modules rather than to one of them.

Constraint or risk	Effect	Handling
Free-tier API rate limits	Hosted free tiers used in development are rate-limited to roughly 15 requests per minute and 1,500 per day for Gemini; the exact figures for whichever service hosts Gemma should be confirmed separately, since they may differ.	The architecture already minimises calls to two per Job Discovery run; caching and the global job store further reduce repeated calls during iteration.
No GPU in the current setup	Fine-tuning and locally hosted large models are outside the scope of the pipeline built in this milestone.	All large-model work is hosted, and the only local model is a small CPU encoder. This was a design input for this milestone rather than an obstacle encountered late.
Vendor model deprecation	Hosted model providers have retired models with short notice in the past, which would break a hard-coded identifier.	All model identifiers are held in environment configuration and can be changed without touching code, across all three modules.
Free-tier PostgreSQL storage	Available storage is limited to roughly 0.3 to 0.5 GB.	The Job Discovery job store is deduplicated by a hash of the normalised posting URL, so storage grows with distinct postings rather than with searches.

5. Implementation Methodology

Each module followed the same loop — implement, run over the development split, inspect output, adjust, re-run.

5.1 Resume Parsing — iteration record and stopping criterion

Two prompt versions were evaluated on a fixed set of five technology resumes with the model held constant. The change made between them was a single explicit instruction requiring experience and project descriptions to be copied word-for-word rather than summarised, added because the first version was observed to paraphrase description text.

Prompt version	Change made	Exact description copy rate	Average latency
Scenario A — original prompt	Baseline extraction instruction; no copying rule stated	20 percent	76.0 seconds
Scenario B — improved verbatim prompt	Adds: copy experience and project descriptions word-for-word; do not summarise, rewrite, improve, or paraphrase	45 percent	96.0 seconds

The explicit copying instruction raised the exact copy rate by 25 percentage points at the cost of 20 seconds of additional latency per resume. The failure it corrected was paraphrasing of free-text description fields, which is precisely the failure that field-level Precision and Recall cannot detect, since a paraphrase is neither a missing field nor a schema violation.

Stopping criterion. Prompt refinement is now stopped on three stated conditions rather than on visible improvement:

- Field extraction is stable, meaning repeated runs over the development split return the same fields for the same input.
- No schema violations occur across the development split.
- No new error classes are introduced relative to the previous prompt version, checked by comparing the returned JSON against the source images for the development records.

5.2 Career Recommendation

Three configuration decisions were taken in this module, each on measured evidence rather than inspection, and are summarised here as the iteration record.

- **Chunk contents.** The indexed occupation chunk was extended from preferred label plus essential skills to preferred label, alternative labels, description, essential skills, and optional skills. Optional skills are present for 2,857 of the 3,039 occupations, so the earlier chunk definition discarded a field that exists for 94 percent of the taxonomy.
- **Skill matching.** Essential and optional skills are now stored as separate metadata fields on each chunk rather than concatenated, so an essential-skill match can be weighted above an optional one. Essential matches carry weight 1.0 and optional matches weight 0.5.
- **Re-ranking rule.** The re-ranking rule specified, sorting on exact skill-match count with cosine similarity only as a tiebreak, was parameterised and swept once labelled data became available. It was replaced with a blended score.

The explanation prompt carries three mode-specific instruction blocks selected at run time: the standard grounded-explanation instruction; an instruction used when the candidate profile lists no skills at all, directing assessment onto job titles and experience and requiring the response to say so; and an instruction used when listed skills produce no overlap with any retrieved occupation, requiring low confidence and an explicit statement that the match is similarity-only. These paths exist so that a sparse or unusual profile produces a qualified answer rather than an empty one.

5.3 Job Discovery and Matching

Four configuration sweeps were run through the evaluation harness: two parameter sweeps on the non-LLM stages and two prompt sweeps on the stages that make LLM calls. Query generation was tested across seven prompt variants on four candidate profiles; the batched judge across six variants on two job batches. Ranking weights were tested across six configurations and the search and crawl parameters across five. Selection in each case is by a weighted composite of quality metrics and latency. Results are reported below.

The judge scores all five shortlisted jobs in a single call rather than five separate calls, which is an efficiency measure rather than a training parameter.

Learning strategy: none. Ranking is computed from BM25, cosine similarity, and configurable weights; no ranking model is trained.

6. Configuration Parameters and Their Selection

There are no hyperparameters in the training sense. The parameters below are the configurable values that govern system behaviour for each module. Each is now labelled by the basis on which it was set: empirical, where the value was selected by a measured sweep across alternatives; or heuristic, where it was set by reasoning about the task and has not yet been varied and measured.

Module	Parameter	Value	Basis
Resume Parsing	Description-copying instruction	Verbatim-copy rule included	Empirical — 2 prompt versions compared, 20 to 45 percent exact copy rate.

Resume Parsing	Parsing model	Gemma 4 31B primary; Gemini 2.5 Flash-Lite fallback	Heuristic — selected on multimodal capability, open weights, and vendor-independence. A two-model comparison was run on five resumes; the sample is too small to change the configuration and the sample is too small to change the configuration.
Resume Parsing	Text-layer probe threshold	100 characters per page	Heuristic — the working corpus contains no page with an extractable text layer, so the threshold is untested on this data
Resume Parsing	Fallback retries	1	Heuristic — cost and latency reasoning; recovery rate of a second retry not measured
Resume Parsing	Pages per call	1	Heuristic — prompt size and per-page validation
Career Recommendation	Skill bonus weight	0.02	Empirical — 8 values swept against MRR and Hit Rate
Career Recommendation	Essential and optional skill weights	1.0 and 0.5	Heuristic — relative weighting set by reasoning; the ratio itself has not been swept
Career Recommendation	Initial retrieval Top-K	20	Heuristic — Hit Rate at 20 of 0.9070 on the annotated set confirms the correct occupation is almost always inside this window, but alternative K values were not compared
Career Recommendation	Final Top-N	5	Heuristic — prompt size and the number of recommendations shown to the user
Career Recommendation	Chunk contents	Label, alternative labels, description, essential and optional skills	Empirical in part — optional skills added after finding they are present for 2,857 of 3,039 occupations
Job Matching	BM25 and embedding weights	0.5 and 0.5	Empirical — 6 configurations compared
Job Matching	Shortlist after hybrid ranking	15	Heuristic — not varied in the sweep
Job Matching	Hybrid and judge weights	0.8 and 0.2	Heuristic — not varied in the sweep
Job Matching	RAG chunks per job at judge time	2	Heuristic — judge prompt size
Job Discovery	Search and crawl parameters	max_results 5, crawl limit 3, concurrency 3, timeout 8000 ms	Empirical — 5 parameter sets compared
All modules	Temperature	0	Heuristic by design — reproducibility is required, so sampling variety has no value in any task here

7. Optimization Methods

Gradient-based optimisers, learning-rate schedules, and gradient clipping do not apply here, since no gradients are computed. The optimisation that did take place was of cost and latency.

7.1 Resume Parsing

No cost or latency optimisation beyond what is already fixed in Section 6.1: one page per call and a single fallback retry. The binding cost for this module is per-page hosted API latency rather than call volume, since exactly one primary call and at most one fallback call are made per page regardless of resume length.

7.2 Career Recommendation

Persistent ESCO index: occupation embeddings are generated once offline and stored in ChromaDB, so no ESCO embedding call is made at request time. This is the module's only distinct cost optimisation, since a candidate query requires exactly one embedding call and one explanation call regardless of how large the ESCO index is.

7.3 Job Discovery and Matching

Optimisation	Effect
Exactly two LLM calls per run	A naive per-item design would issue one extraction call per discovered job and one judge call per shortlisted job, an estimated 45 or more calls per run. Making extraction deterministic and batching the judge reduces this to two calls regardless of how many jobs are found.
Zero-LLM extraction	Structured fields are read from schema.org JobPosting JSON-LD or page metadata rather than from a model call. This is faster, more reliable, and free per job.
Global deduplicated job store	Every posting URL reduces to a stable key. A posting is crawled, parsed, and embedded once, then reused by every subsequent search from any candidate until it goes stale or expires.
Local embedding model	bge-base-en-v1.5 runs on CPU, so candidate and job embeddings cost nothing per call and are unaffected by API rate limits.
Redis result cache	SearXNG results are cached for one hour, so repeated or similar queries during development do not re-issue searches.

7.4 Tuning Experiments

7.4.1 Resume Parsing

A further experiment held the improved verbatim prompt fixed and varied the parsing model across the same five technology resumes. This was run to quantify how much of the extraction behaviour is attributable to the prompt and how much to the model. Gemma 4 31B remains the primary parser with Gemini 2.5 Flash-Lite as the fallback and repair path.

Model	Successful runs	Average latency	Exact copy rate
Gemma 4 31B (current primary parser)	5 of 5	96.0 seconds	33.3 percent
Gemini 3.5 Flash (alternative tested)	5 of 5	14.3 seconds	91.7 percent

Exact copy rate here is computed over resumes for which the model returned at least one description. The alternative model was approximately 6.7 times faster on this sample and returned more descriptions matching the source text word-for-word. The sample is five resumes, which is too small to support a model change on its own, and the comparison does not cover the other extracted fields or the schema-validity and fallback behaviour on which the current parser was selected. No configuration change is made on this evidence.

7.4.2 Career Recommendation

The module is now evaluated against the 86-record annotated resume gold set. Because that set carries a resume category rather than an ESCO occupation label, each of the 43 categories was mapped to a set of acceptable ESCO occupations, 134 mappings in total, and a recommendation counts as correct when it matches any occupation in the set for that resume's category.

Design of the sweep. The re-ranking step combines two signals: cosine similarity from retrieval, and exact overlap between the candidate's skills and the occupation's ESCO skill list. The question the sweep answers is how much influence the skill-overlap signal should carry relative to similarity. This was parameterised as a single skill-bonus weight, and the whole range was measured on the 86-record annotated set, from a pass-through at one end through to skill-dominant settings at the other. Retrieval alone provides the reference line the step must beat. Reference line — retrieval alone, Top-K 20, on all 86 records: MRR 0.6018, Hit Rate at 1 of 0.4651, Hit Rate at 3 of 0.6860, Hit Rate at 5 of 0.7791.

Weight sweep. Semantic similarity acts as the primary signal, with normalised skill overlap contributing a bounded bonus. The bonus weight was swept across eight values.

Skill bonus weight	MRR	Hit Rate at 1	Hit Rate at 3	Hit Rate at 5
0.00 (pass-through)	0.5849	0.4651	0.6860	0.7791
0.02 — selected	0.5965	0.4884	0.6744	0.7907
0.05	0.5866	0.4767	0.6395	0.7907
0.08	0.5457	0.4070	0.6279	0.7907
0.10	0.5328	0.3953	0.6279	0.7674
0.15	0.4973	0.3372	0.6047	0.7674
0.25	0.4866	0.3256	0.5930	0.7674
0.40	0.4843	0.3256	0.5930	0.7558

At weight 0.00 the step is a pass-through and reproduces the retrieval ordering exactly, which confirms the implementation is correct and establishes the floor the step must beat. Weight 0.02 improves on that floor on every reported measure and was selected. Metrics decline monotonically above 0.05.

The shape of the curve is the substantive finding of this sweep. Exact skill matching contributes usefully in this module only at low weight, as a tiebreaker between occupations of comparable similarity, and not as a primary ranking signal. The reason is measurable: resume skill text and ESCO skill labels draw on different vocabularies, so exact string overlap fires infrequently on real resumes, and when it does fire it is often on a generic term. A skill-dominant setting therefore allows one incidental match to outrank an occupation with strong semantic similarity and no exact match. Such a setting, sorting on skill-match count with similarity only as a tiebreak, sits at that end of this range; its measured position is shown below.

Held-out confirmation. The selected configuration was then run on the 52-resume held-out split.

Stage (held-out test split, 52 resumes)	MRR	Hit Rate at 1	Hit Rate at 3	Hit Rate at 5
Retrieval only (Top-K 20)	0.6437	0.5192	0.7500	0.8077
Re-ranking at selected weight 0.02	0.6359	0.5192	0.7500	0.8269

On held-out data the selected configuration matches retrieval on Hit Rate at 1 and Hit Rate at 3, improves Hit Rate at 5 from 0.8077 to 0.8269, and holds MRR within 0.008 of retrieval. The per-record diagnostic is 1 case improved, 2 cases worsened, and 3 cases lost to the truncation from 20 candidates to 5. Recall at 5 is 0.4439 and Precision at 5 is 0.2808 on the same run. The held-out figures are at or above the corresponding figures on the full annotated set, which is consistent with the selected weight not being overfitted to it.

7.4.3 Job Discovery and Matching

Four sweeps were run through the prompt for evaluation harness: two parameter sweeps on the non-LLM stages and two prompt sweeps on the stages that do make LLM calls. Selection in every case is by a weighted composite score that combines quality metrics with latency.

Ranking stage — 6 weight configurations, 86 candidates with 10 jobs each, real embeddings and BM25.

Configuration	Composite	Precision at 5	NDCG at 10	Spearman	Latency (ms)	Pass rate
balanced_50_50 — best	0.8977	0.8486	0.9795	0.8767	226	94.6%
embedding_heavy	0.7670	0.8588	0.9793	0.8861	4145	0%
bm25_only	0.7656	0.8512	0.9793	0.8995	4239	0%
current_default (0.4 / 0.3)	0.7593	0.8518	0.9791	0.8812	4241	0%
embedding_only	0.7518	0.8419	0.9706	0.8727	4201	0%
bm25_heavy	0.7503	0.8535	0.9789	0.8796	4500	0%

Quality is near-identical across all six configurations, in the range 0.85 to 0.90 on the ranking metrics. The separation is almost entirely latency: 226 milliseconds for the winner against more than 4,000 for every alternative. The composite weights latency at 15 percent, which is what decides this leaderboard. The practical conclusion is that the BM25 and embedding weights, recorded above, are not a sensitive parameter for ranking quality on this data, and an equal 0.5 and 0.5 split was selected.

Search and crawl stage — 5 parameter sets across 5 test queries.

Configuration	Composite	Coverage	Crawl success	Latency (ms)
narrow_fast — best	0.4500	1.0	0.50	2703
high_concurrency	0.4152	1.0	0.45	6868
current_default	0.4041	1.0	0.40	8194
wide_net_slow	0.3618	1.0	0.067	16030
low_concurrency_safe	0.3433	1.0	0.267	26604

Coverage is saturated at 1.0 for every configuration, so the discriminating measures are crawl success rate and latency. The best configuration crawls successfully on half of attempted URLs; the widest configuration succeeds on under 7 percent while taking six times as long. Crawl success is therefore an open reliability problem and is recorded among the implementation statistics, not presented as solved.

Query-generation prompt — 7 variants across 4 candidate profiles, 28 live calls.

Prompt variant	Composite	Schema score	Diversity score	Latency (ms)	Pass rate
boolean_operators — best	0.5102	1.0000	0.8010	19	100%
roleplay	0.4306	1.0000	0.8010	26	100%
chain_of_thought	0.4255	1.0000	0.8010	27	100%
few_shot	0.3986	1.0000	0.8010	29	100%
concise	0.3800	1.0000	0.8010	31	100%
baseline	0.3646	1.0000	0.8010	32	100%
minimal	0.3602	1.0000	0.8010	33	100%

All seven variants return schema-valid and sufficiently diverse output, so schema and diversity scores are identical and the winner is decided on latency alone. The winning variant differs from the baseline by instructing the model to use search-engine operators such as site:, quoted phrases,

and OR, mixed with plain-text queries, and by showing two worked examples of that pattern, rather than describing four abstract query categories.

Batched judge prompt — 6 variants across 2 job-batch cases, 12 live calls.

Prompt variant	Composite	Schema score	Calibration score	Latency (ms)	Pass rate
comparative — best available	0.2750	0.5000	0.5000	21572	0%
narrative	0.2060	0.5000	0.5000	28082	0%
baseline	-0.0668	0.0000	0.0000	27872	0%
weighted_criteria	-0.0811	0.0000	0.0000	29219	0%
strict_rubric	-0.0942	0.0000	0.0000	30450	0%
chain_of_thought	-0.1500	0.0000	0.0000	35716	0%

This sweep is reported as a negative result. The best available variant reaches only 0.5 on schema validity and 0.5 on calibration, and no variant achieves a passing rate. Two concrete failure modes were captured by the validators: the baseline prompt returned text that was not JSON at all, and the selected comparative prompt prepended visible reasoning before the JSON array on one of two cases, breaking the parser, while a second case produced valid output that exceeded the 20-second latency ceiling. The comparative variant was selected because its rank-then-score instruction yields internally consistent probability values when it does return clean JSON, which every other variant fails to do at all. It is the least unreliable option available, not a working one, and this is recorded among the current limitations.

Validation mechanism. All grading in these sweeps is deterministic and rule-based rather than model-graded, specifically to avoid a second LLM call per evaluated output. Query generation is checked for valid JSON array structure, exact expected length, all-string elements, a uniqueness ratio above 0.7, and a token-overlap diversity score above 0.4. The judge output is checked for a valid JSON array of expected length, an interview probability within 0 to 100, a recommendation drawn from a closed set, a non-empty reason string, and a required-skills list; a separate calibration check requires the recommendation to be numerically consistent with the probability. Both sweeps also apply a hard latency ceiling as a pass or fail condition.

8. Regularization and Reliability Controls

Dropout, weight decay, and label smoothing do not apply, as there are no weights to regularise. The equivalent concern here is that a pretrained model may produce output that is fluent but unsupported by the source document.

8.1 Resume Parsing

- **Schema constraints:** every call is bound to `parsed_resume_schema.json`, and validation is a hard gate before any profile is persisted or passed downstream.
- **Visible-only extraction:** the prompt forbids inference or guessing. Missing scalar fields are stored as null and missing lists as empty, rather than being filled from plausible assumption.
- **Duplicate-safe splits:** perceptual near-duplicate groups in the resume corpus are confined to a single split, so a visually identical resume cannot appear on both sides of an evaluation boundary.
- **Category-balanced sampling:** the gold set takes two resumes from each of the 43 canonical categories, so that the largest category, which holds roughly 12 times as many resumes as the smallest, cannot dominate the evaluation.
- **Prompt-injection handling:** resume text is treated as untrusted data, delimited clearly, and cannot alter the system instruction.

8.2 Career Recommendation

- **Schema constraints:** the explanation model's output is bound to a Pydantic response schema, validated before being returned.
- **Evidence grounding:** the explanation model may only discuss occupations present in the retrieved set, and may only name skill gaps drawn from that occupation's own ESCO skill list. Every output URI is validated against the retrieved set.

8.3 Job Discovery and Matching

- **Schema constraints:** the batched judge call returns structured output, validated before being accepted.
- **Permissive filtering:** the hard filter excludes a posting only when it states a requirement the candidate explicitly fails. Vague postings pass through, because a wrongly excluded job cannot be recovered later in the pipeline.
- **Prompt-injection handling:** job descriptions are treated as untrusted data, delimited clearly, and cannot alter the system instruction.

9. Development Progress

There are no loss curves or accuracy-against-epoch plots to report, since nothing is trained. What follows is the state of each module at the end of this milestone.

9.1 Resume Parsing

Status: working prototype. The extraction path runs end to end. A resume is routed by the text-layer probe, rendered page by page where required, sent to the parser with the schema prompt, and the returned JSON is validated, merged across pages, normalised, and deduplicated. The fallback retries path and the partial-profile-with-review-flags path both function.

9.2 Career Recommendation

Status: working prototype. The offline stage is complete: all 3,039 ESCO occupations have been converted to occupation-profile chunks, embedded, and written to the persistent ChromaDB index. The inference stage runs: a candidate profile is flattened into a semantic query, embedded, matched against the index, re-ranked deterministically, and passed to the explanation model, which returns schema-validated output.

9.3 Job Discovery and Matching

Status: working prototype. All seven pipeline stages are implemented and run in sequence against live search. Query generation, SearXNG search with Redis caching, Crawl4AI extraction on a job-store miss, regex hard filtering, hybrid BM25 and embedding ranking, and the batched judge call all function, and the ranked shortlist of five jobs is returned with scores and rationale. The global job store correctly skips crawling for postings already held.

9.4 End-to-End Worked Example

9.4.1 Resume Parsing Module

Input: a single-page image-only resume. Output: schema-validated candidate profile JSON.

Abridged extract:

```
{ "contact": { "name": "David Clark", "location": "Columbus, Ohio", "links": ["linkedin.com"] },
  "skills": ["SQL", "T-SQL", "Information Technology", "Data Migration", "Audit", "Complex SQL Development", "T-SQL Procedures Coding"],
  "education": [{ "degree": "Bachelor's Degree", "field": "Information Technology", "institution": "The Ohio State University", "start_year": "2003", "end_year": "2007" }],
  "experience": [{ "job_title": "Senior SQL Developer", "company": "JP Morgan Chase", "location": "Columbus, OH", "start_date": "2017", "end_date": "Ongoing", "current_role":
```

```
true, "description": "Directed teams in designing and implementing complex SQL solutions ..."}},
"projects": [], "certifications": [{ "name": "Microsoft Certified: SQL Server 2016 Development",
"issuer": "Microsoft", "year": null }], "job_titles": ["Senior SQL Developer", "SQL Developer", "Data
Migration Specialist"] }
```

Three behaviours specified among the reliability controls are visible in this output. The projects list is empty rather than populated by inference, because the source resume contains no projects section. The certification year is null rather than guessed. No email or telephone number appears, in line with the retention rule.

9.4.2 Career Recommendation Module

Input: the candidate profile above. The profile is flattened into a query, embedded, matched against the 3,039-occupation ESCO index, re-ranked, and passed to the explanation model. Deterministic ranking before the explanation call, for a data-analyst profile, showing the two-part score:

Occupation	Matched essential skills	Matched optional skills	Similarity
data analyst	Python, SQL, data visualisation, statistics	—	high
data scientist	Python, SQL, statistics	—	high
statistical assistant	statistics	—	medium
financial analyst	statistics, SQL	—	medium
database developer	SQL	—	medium

Explanation-model output for the same run, abridged to three of five entries. Each entry carries the confidence band and the evidence it is grounded in:

- data analyst — confidence high. Direct match: the candidate currently works as a Junior Data Analyst and holds core technical skills in Python, SQL, and data visualisation. Matched evidence: Python, SQL, data visualisation, statistics.
- data scientist — confidence medium. Relevant technical foundation in Python, SQL, and statistics from data-analysis work, making them a potential fit for a junior transition. Matched evidence: Python, SQL, statistics.
- database developer — confidence low. Demonstrates database querying capability through SQL, but lacks the broader software engineering and database architecture skills required for a dedicated developer role. Matched evidence: SQL.

The confidence bands track the deterministic evidence rather than being asserted by the model: five matched skills yield high, one matched skill yields low. The low-confidence entry states what is missing rather than overselling the match. Every returned ESCO URI in this run passed validation against the retrieved set, so no entry was dropped by the anti-hallucination guard.

9.4.3 Job Discovery and Matching Module

Per-candidate ranking metrics from the evaluation harness, for one candidate against a ten-job pool: Precision@5 of 1.0, NDCG@10 of 0.9679, and Spearman correlation of 0.7091, at BM25 weight 0.5 and embedding weight 0.5.

Discovery stage for one query:

```
{"query": "Data Analyst jobs Bangalore", "mode": "real", "unique_urls_found": 5,
"urls_attempted_crawl": 3, "crawled_success": 0, "crawl_success_rate": 0.0, "total_latency_ms":
13508.4 }
```

Search returned five unique URLs and three were attempted, but none crawled successfully on this query. The pipeline did not fail; the failed URLs were skipped. Crawl success across the full sweep

averages 0.50 on the selected configuration, which is the figure quoted in the implementation statistics.

Sample generated search queries from the selected query-generation prompt, for a data-analyst profile: Data Analyst jobs; SQL Python analyst; fintech data analyst; Business Analyst Bangalore; Analytics Engineer remote; junior data analyst jobs.

The resulting shortlist as presented to the user, showing the match percentage, recommendation band, matched and missing criteria, and the rationale for each job. Output of this quality is not yet produced reliably; the judge prompt's reliability is recorded under current limitations.

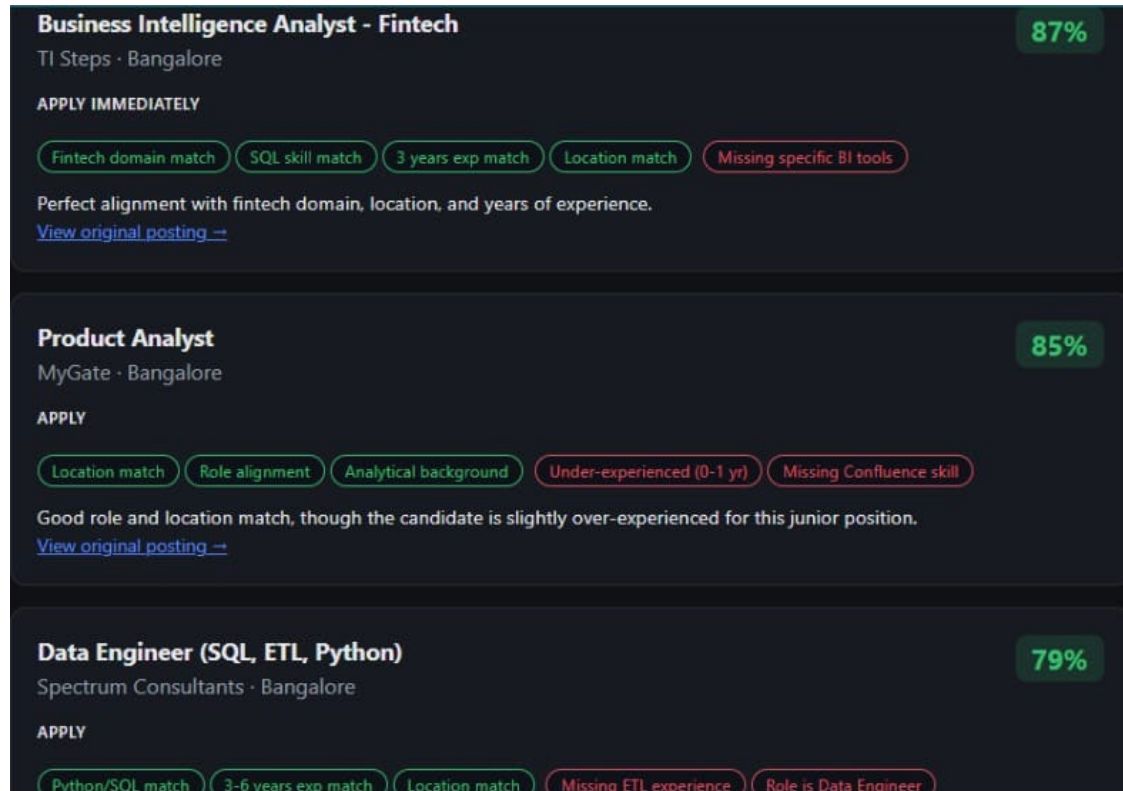


Figure 1. Job Discovery and Matching output for a data-analyst candidate.

10. Final Configuration Selection

Because no models are trained, there are no checkpoints to compare and no best epoch to select. What is fixed is a configuration bundle per module, which is the artefact the system currently runs on.

10.1 Resume Parsing

Element	Fixed value
Model identifiers	Gemma 4 31B (primary parser); Gemini 2.5 Flash-Lite (fallback and repair)
Prompts and schemas	Resume extraction prompt bound to parsed_resume_schema.json
Thresholds	Text-layer probe 100 characters per page; one fallback retry; temperature 0

10.2 Career Recommendation

Element	Fixed value
Model identifiers	BAAI/bge-base-en-v1.5 (768 dimensions, local CPU); Gemini 2.5 Flash-Lite (explanation)
Prompts and schemas	Career explanation prompt bound to its Pydantic response schema
Retrieval parameters	Top-K 20; Top-N 5
Thresholds	Temperature 0

10.3 Job Discovery and Matching

Element	Fixed value
Model identifiers	Gemini 2.5 Flash-Lite (query generation); Gemma 4 31B (judge); BAAI/bge-base-en-v1.5 (embedding)
Prompts and schemas	Query-generation prompt; batched judge prompt with structured output
Retrieval parameters	Shortlist 15; 2 RAG chunks per job at judge time
Scoring weights	BM25 0.5, embedding 0.5, normalisation 0.7; hybrid 0.8, judge 0.2
Thresholds	Temperature 0

11. Challenges Encountered

11.1 Resume Parsing

Challenge	Effect	How it was handled
The gold set is provisional	Only 15 of the 86 gold records were verified by three team members against the source images; the remaining 71 were AI-drafted and not individually checked. Reported field-level results will carry the accuracy of the annotations themselves.	The limitation is stated rather than hidden. Verification of the remaining records is scheduled before the held-out evaluation.
Sparse fields in the corpus	Projects appear in 5 of 86 gold records and certifications in 27, so per-field results for these two fields will rest on very few examples.	Per-field metrics will be reported with support counts, and no overall score will be quoted without them.

11.2 Career Recommendation

Challenge	Effect	How it was handled
No directly labelled candidate-to-occupation data	ESCO occupation labels do not exist for the resume corpus, so retrieval cannot be scored against a definitive answer key	Each of the 43 resume categories was mapped to a set of acceptable ESCO occupations, 134 mappings in total, and a recommendation is counted correct when it matches any occupation in that

Challenge	Effect	How it was handled
		set. The mapping is a proxy: it was generated by keyword matching against ESCO titles and reviewed, several categories map to overlapping occupations which caps achievable Precision, and four categories currently return no match.

11.3 Job Discovery and Matching

Challenge	Effect	How it was handled
Crawl reliability	Some job boards render poorly or block automated access, and postings without JSON-LD yield weaker structured fields.	A failed crawl skips that URL rather than failing the run. Restricted pages are excluded rather than bypassed.

11.4 Cross-cutting constraints

Free-tier API rate limits, the absence of a GPU in the current setup, and vendor model deprecation affect all three modules rather than one.

11.5 Quantitative Implementation Statistics

Measure	Value	Notes
ESCO occupations indexed	3,039	One chunk per occupation; all unique URIs, no empty documents
Occupations carrying optional skills	2,857 of 3,039	94 percent; this field was absent from the earlier chunk definition
Mean indexed document length	approximately 1,880 characters	Maximum 5,318 characters
ESCO index build time	minutes, on CPU	Local embedding model; one offline pass, no request-time embedding of the taxonomy
Resumes processed in evaluation	86 annotated records	34 development and 52 held-out test
Career Recommendation evaluation runtime	11.7 seconds for 86 profiles; 6.8 seconds for 52	Retrieval and re-ranking only; the explanation call is not exercised by the metric harness
Career Recommendation failures during evaluation	0 of 86	No profile failed to produce a ranked list
Resume parsing latency, current parser	96.0 seconds average	On five resumes with the verbatim-copy prompt. An alternative model averaged 14.3 seconds on the same sample
Job Matching golden dataset	86 candidates x 10 jobs	Same 86 records as the resume gold set
Job Matching ranking latency, best configuration	226 ms	Against more than 4,000 ms for every rejected configuration
Job Matching pass rate, best configuration	94.6 percent	Only configuration in the sweep to pass

Job Discovery test queries	5	Across 5 parameter configurations
Job Discovery latency, best configuration	2,703 ms	Against 26,604 ms for the slowest configuration
Crawl success rate, best configuration	0.50	Open reliability issue; 0.067 for the widest configuration
Query-generation latency	19 ms, best prompt variant	Range 19 to 33 ms across 7 variants
Batched judge latency	21,572 ms, best prompt variant	Exceeds the 20-second ceiling; open defect
Live LLM calls made during prompt sweeps	40	28 query-generation, 12 judge

Figures not yet available and recorded as gaps rather than estimated: average API latency aggregated across all three modules; embedding generation time as a batch figure for the job pipeline, where only per-call latency is logged; and average jobs retrieved per candidate outside the golden dataset.

12. Summary and Next Steps

12.1 Summary

The system comprises three working prototype modules. Resume Parsing extracts a validated candidate profile from an image-only resume; Career Recommendation has the full ESCO taxonomy embedded and indexed and returns grounded occupation recommendations; Job Discovery and Matching runs its seven-stage pipeline against live search and returns a ranked shortlist of five jobs with scores and rationale.

No training took place, and none was planned. All models are pretrained and used zero-shot or as frozen encoders, for the reasons given above. The configurable surface of the system, meaning prompts, schemas, retrieval parameters, and scoring weights, is set out in the configuration and final configuration sections.

12.2 Current limitations

The batched judge prompt does not yet return reliably parseable output; the best of six variants tested reaches 0.5 on schema validity and 0.5 on calibration. Crawl success reaches 0.50 on the selected configuration. The Career Recommendation gold labels map resume categories to sets of acceptable ESCO occupations rather than to directly labelled occupations, and four categories currently return no match. The local encoder truncates at 512 tokens, so the longest occupation documents are not embedded in full. Of the 86 resume gold records, 15 were verified against source images. Several configuration values remain heuristic and are labelled as such in the configuration table.

13. Team Consent

Name	Roll Number	Initial
Gaurav Kumar	22f1001105	GK
Dev Gupta	22f2000888	DG
Abhinav Ohri	24f1002064	AO
Pranav N	22f2000117	PN
Praveena N	22f3001454	PN